

One-to-One Table Matching Without Shared Keys; the Argentinean subset.

Biodiversity Data Integration

2025-12-07

Contents

1	Introduction	2
2	Load Required Libraries	2
3	Data importation	3
4	Join the submitted and the received table by id_CC, and compare corresponding fields between both tables	4
5	Data Preparation	8
5.1	Filter Source Data	8
5.2	Standardization Function	9
5.3	Apply Standardization	10
6	Matching Strategy	10
6.1	Define Matching Keys	10
6.2	Strictly Conservative Matching Function	11
6.3	Execute Matching	13
7	Results and Diagnostics	14
7.1	Overall Matching Success	14
7.2	Match Status Distribution	14
7.3	Match Quality for Successfully Matched Rows	16
7.4	Keys Required for Matching	16
7.5	Specific Key Combinations	19
7.6	Core vs Extra Keys Usage	19
7.7	Uniqueness Validation	20
7.8	Ambiguous Matches Analysis	22
7.9	No Match Analysis	22
7.10	Unmatched Rows Requiring Review	23

7.11 Matched, rowwise field-by-Field Comparison	24
7.12 Matched, overall field-by-Field Comparison	28
8 Export Results	32
8.1 Matched Records	32
8.2 Good-to-bad match ratio	34
8.3 Unmatched Records for Manual Review	34
8.4 Ambiguous Matches for Investigation	35
9 Summary Statistics Table	35
10 Session Information	36
11 Conclusion	37

1 Introduction

Aquí se documenta cómo se vincularon los comentarios y revisiones de Daniela Milano y Juan Pablo Barriga sobre las ocurrencias reportadas en territorio Argentino. Se implementó la misma aproximación descrita para el total del set de datos (ver `Matching_full_dedup_dataset_with_comments_2025-12-06.Rmd`).

Este documento describe el método empleado para reestablecer el vínculo uno-a-uno entre la tabla de comentarios y revisiones (`table1`) y la de la base de datos (`table2`) de modo de poder ejecutar dichos cambios, asegurando correspondencias precisas y completamente trazables.

Principios clave del flujo de trabajo:

- Cada fila de `table1` debe coincidir de manera exacta con una única fila de `table2`.
- Las comparaciones se realizan exhaustivamente contra todas las filas de la otra tabla.
- Las coincidencias se establecen mediante combinaciones incrementales de campos clave previamente armonizados.
- El orden estratégico de estos campos favorece coincidencias correctas, conservadoras y no ambiguas.

2 Load Required Libraries

```
setwd("C:/Users/crist/Documents/GitHub/PEW-ProAp_Atlas_Ictiogeografico_2025/Atlas_Ictio_project/")
rm(list=ls())

library(dplyr)
library(stringr)
library(ggplot2)
library(knitr)
library(tidyr)
```

3 Data importation

Current database

```
# Load data to start from here

X.filtered.dedup <- readRDS("./outputs_global_analyses/dataset_before_Remocion_manual_2025-12-03.rds")
```

Database version submitted to Daniela Milano and Juan Pablo Barriga

```
DB.submitted <- read.csv("./input_parameters/AtlasIctio2025_Datos_DwC_deInteres_norepetidos.csv", sep = ",")

# Merge (coalesce) two columns that were separated only due to a typo.

DB.submitted <- DB.submitted %>%
  mutate(
    across(c(bibliographicCitation, bibliographicCitation),
      ~ str_trim(.) %>% na_if("") %>% na_if("NA")),
    bibliographicCitation = coalesce(bibliographicCitation, bibliographicCitation)
  ) %>% select(-bibliographicCitation)
```

Database received back with comments

```
# CM.Ar <- read.csv("./input_parameters/AtlasIctio_COMMENTS_DMilano_JPBarriga_Argentina.csv", sep = ",", c

CM.Ar <- readxl::read_xlsx("./input_parameters/AtlasIctio2025_Datos_DwC_deInteres_norepetidos DM.xlsx", na

CM.Ar <- CM.Ar %>%
  mutate(eventDate = if_else(
    grepl("[0-9]+$", eventDate), # only digits + likely Excel serial
    as.character(as.Date(as.numeric(eventDate), origin = "1900-01-01")), #origin = "1899-12-30"
    eventDate
  ))

## Fix character encoding-related issues.
CM.Ar <- CM.Ar %>%
  mutate(across(
    .cols = where(is.character) &
      !any_of("correccion") &
      !starts_with("."),
    .fns = ~ iconv(., from = "UTF-8", to = "Windows-1252", sub = "byte")
  ))

# CM.Ar %>% select(locality, correccion, starts_with(".")) %>% filter(!is.na(correccion))

# Merge (coalesce) two columns that were separated only due to a typo.

CM.Ar <- CM.Ar %>%
  mutate(
    across(c(bibliographicCitation, bibliographicCitation),
      ~ str_trim(.) %>% na_if("") %>% na_if("NA")),
    bibliographicCitation = coalesce(bibliographicCitation, bibliographicCitation)
```

```

) %>% select(-bibliographicCitation)

# Remove "NUEVOS" and sort by id_CC
CM.Ar <- CM.Ar %>% filter(id_CC != "NUEVO") %>%
  arrange(as.numeric(id_CC))

# Rename the correccion column to make it compatible with code below
CM.Ar <- CM.Ar %>% mutate(Comment = correccion) %>%
  # relocate(Comment, .before = .duplicate) %>%
  select(-correccion)

# Keep only rows with comments in another object
edits.Ar <- CM.Ar %>%
  # filter(rowSums(!is.na(across(c(Comment, starts_with(".")))))) &
  # across(c(Comment, starts_with(".")), ~ .x != "") > 0 )
  filter(!is.na(Comment))

baseline <- DB.submitted #>% select(-starts_with("."), -Duplicate)

edits.Ar.matched <- baseline %>% right_join(edits.Ar, by = "id_CC", keep = F) %>%
  select(id_CC, starts_with("canonicalName"), Comment)

# LOTS OF ROWS IN Ar WERE NOT FOUND IN CM
# rm(edits.Ar.fixed, edits.Ar, baseline, edits.Ar.matched)

cat("The proportion of missmatching canonicalName is:",
    100 * sum(edits.Ar.matched$canonicalName.x != edits.Ar.matched$canonicalName.y, na.rm = T) /
    nrow(edits.Ar.matched), "%" )

## The proportion of missmatching canonicalName is: 65.41787 %

```

4 Join the submitted and the received table by id_CC, and compare corresponding fields between both tables

Note: Self-comparisson produces perfect or almost perfect match in datasetName, coordinates, canonicalName, locality and bibliographicCitation.

```

assess_field_correspondence <- function(baseline, edits, join_key = "id_CC",
                                         candidate_fields, plot = TRUE) {

  # Join tables
  joined <- baseline %>%
    right_join(edits, by = join_key, suffix = c(".baseline", ".edits"), keep = FALSE)

  # Initialize results data frame
  results <- data.frame(
    field = character(),
    n_total = integer(),
    n_matches = integer(),
    n_mismatches = integer(),
    n_both_na = integer(),
    n_only_baseline_na = integer(),

```

```

  n_only_edits_na = integer(),
  pct_matches = numeric(),
  pct_mismatches = numeric(),
  pct_any_na = numeric(),
  stringsAsFactors = FALSE
)

# Assess each field
for (field in candidate_fields) {

  col_baseline <- paste0(field, ".baseline")
  col_edits <- paste0(field, ".edits")

  # Check if columns exist
  if (!col_baseline %in% names(joined) || !col_edits %in% names(joined)) {
    warning(paste("Field", field, "not found in one or both tables. Skipping."))
    next
  }

  baseline_vals <- joined[[col_baseline]]
  edits_vals <- joined[[col_edits]]

  # Calculate statistics
  n_total <- nrow(joined)

  both_na <- is.na(baseline_vals) & is.na(edits_vals)
  only_baseline_na <- is.na(baseline_vals) & !is.na(edits_vals)
  only_edits_na <- !is.na(baseline_vals) & is.na(edits_vals)
  both_present <- !is.na(baseline_vals) & !is.na(edits_vals)

  matches <- both_present & (baseline_vals == edits_vals)
  mismatches <- both_present & (baseline_vals != edits_vals)

  n_matches <- sum(matches, na.rm = TRUE)
  n_mismatches <- sum(mismatches, na.rm = TRUE)
  n_both_na <- sum(both_na, na.rm = TRUE)
  n_only_baseline_na <- sum(only_baseline_na, na.rm = TRUE)
  n_only_edits_na <- sum(only_edits_na, na.rm = TRUE)

  # Add to results
  results <- rbind(results, data.frame(
    field = field,
    n_total = n_total,
    n_matches = n_matches,
    n_mismatches = n_mismatches,
    n_both_na = n_both_na,
    n_only_baseline_na = n_only_baseline_na,
    n_only_edits_na = n_only_edits_na,
    pct_matches = round(100 * n_matches / n_total, 1),
    pct_mismatches = round(100 * n_mismatches / n_total, 1),
    pct_any_na = round(100 * (n_both_na + n_only_baseline_na + n_only_edits_na) / n_total, 1),
    stringsAsFactors = FALSE
  ))
}

# Sort by mismatch rate

```

```

results <- results %>% arrange(desc(pct_mismatches), pct_matches)

# Create plot if requested
if (plot && nrow(results) > 0) {

  # Prepare data for plotting
  plot_data <- results %>%
    select(field, n_matches, n_mismatches, n_both_na,
           n_only_baseline_na, n_only_edits_na) %>%
    pivot_longer(cols = -field, names_to = "category", values_to = "count") %>%
    mutate(
      category = factor(category,
                        levels = c("n_matches", "n_mismatches", "n_both_na",
                                   "n_only_baseline_na", "n_only_edits_na"),
                        labels = c("Matches", "Mismatches", "Both NA",
                                   "Baseline NA", "Edits NA")),
      field = factor(field, levels = rev(results$field))
    )

  # Create stacked barplot
  p <- ggplot(plot_data, aes(x = field, y = count, fill = category)) +
    geom_bar(stat = "identity", position = "stack") +
    coord_flip() +
    scale_fill_manual(
      values = c("Matches" = "#2ecc71",
                 "Mismatches" = "#e74c3c",
                 "Both NA" = "#95a5a6",
                 "Baseline NA" = "#f39c12",
                 "Edits NA" = "#e67e22")
    ) +
    labs(
      title = "Field Correspondence Assessment",
      subtitle = paste("Comparing", nrow(joined), "records"),
      x = "Field",
      y = "Number of Records",
      fill = "Status"
    ) +
    theme_minimal() +
    theme(
      legend.position = "bottom",
      axis.text.y = element_text(size = 10)
    )

  print(p)
}

# Return results
list(
  summary = results,
  joined_data = joined
)
}

# Usage example:
candidate_fields <- c("canonicalName", "decimalLatitude", "decimalLongitude",
                     "year", "eventDate", "locality",

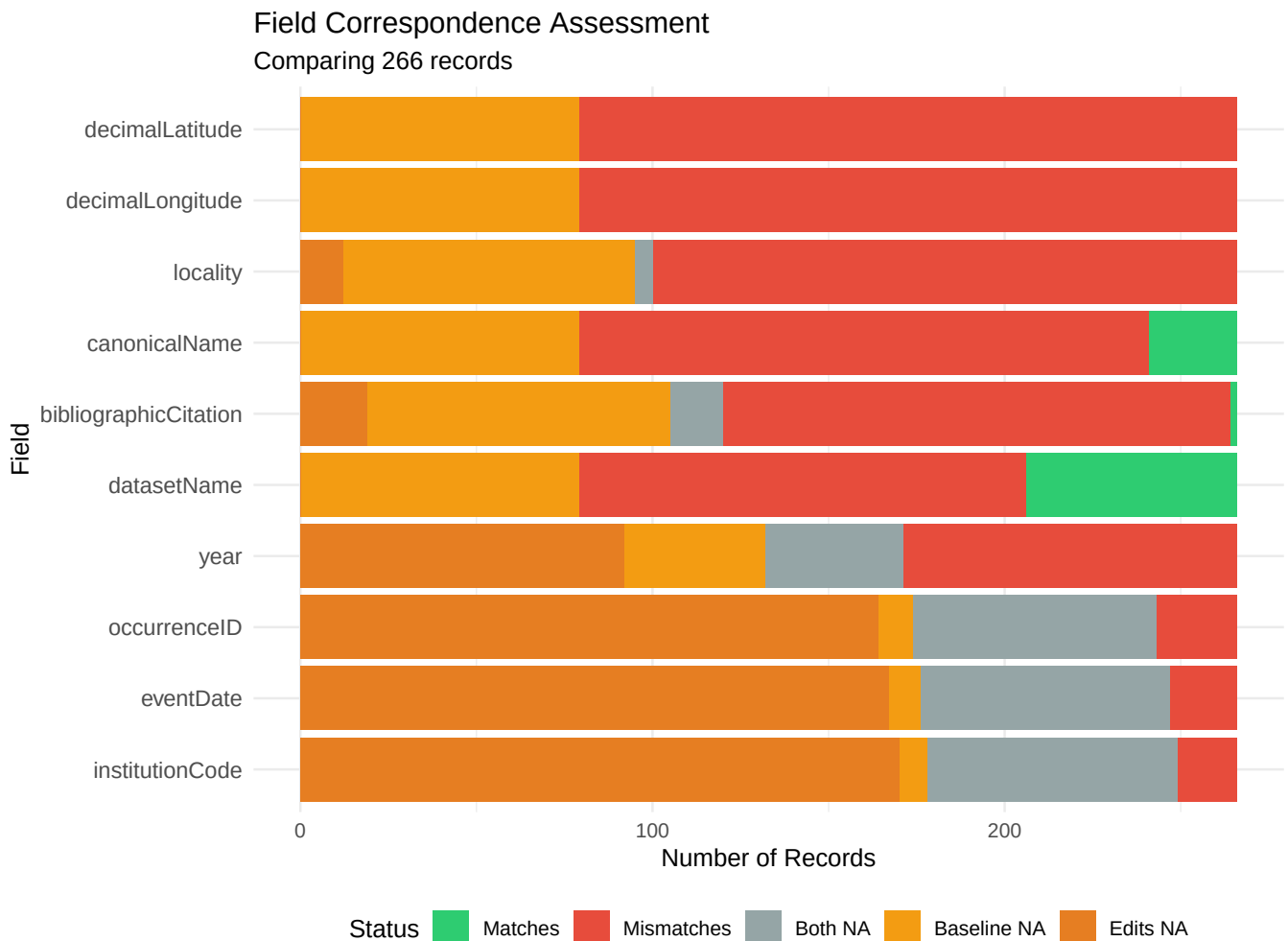
```

```

      # "samplingProtocol", "individualCount", "catalogNumber",
      "datasetName",
      "bibliographicCitation", "institutionCode", "occurrenceID")

assessment <- assess_field_correspondence(
  baseline = DB.submitted, # DB.submitted,
  edits = edits.Ar[!(edits.Ar$institutionCode %in% "FO"),], # Exclude Fundacion Oga
  join_key = "id_CC",
  candidate_fields = candidate_fields,
  plot = TRUE
)

```



```

# View summary table
kable(assessment$summary, caption = "Field Correspondence Summary")

```

Table 1: Field Correspondence Summary

field	n_total	n_matches	n_mismatches	both_na	only_baseline	only_edits	both_na_match	both_na_mismatch	both_na_na
decimalLatitude	266	0	187	0	79	0	0.0	70.3	29.7
decimalLongitude	266	0	187	0	79	0	0.0	70.3	29.7
locality	266	0	166	5	83	12	0.0	62.4	37.6
canonicalName	266	25	162	0	79	0	9.4	60.9	29.7
bibliographicCitation	266	2	144	15	86	19	0.8	54.1	45.1

field	n_total	n_matches	n_mismatches	both_na	only_baseline	only_edits	pct_matches	pct_mismatches	pct_any_na
datasetName	266	60	127	0	79	0	22.6	47.7	29.7
year	266	0	95	39	40	92	0.0	35.7	64.3
occurrenceID	266	0	23	69	10	164	0.0	8.6	91.4
eventDate	266	0	19	71	9	167	0.0	7.1	92.9
institutionCode	266	0	17	71	8	170	0.0	6.4	93.6

```
# Access specific results
cat("\nFields with highest mismatch rates:\n")
```

```
##
## Fields with highest mismatch rates:
```

```
assessment$summary %>%
  arrange(desc(pct_mismatches)) %>%
  select(field, pct_mismatches, n_mismatches) %>%
  head(5)
```

```
##               field pct_mismatches n_mismatches
## 1 decimalLatitude      70.3           187
## 2 decimalLongitude      70.3           187
## 3 locality             62.4           166
## 4 canonicalName        60.9           162
## 5 bibliographicCitation 54.1           144
```

```
CM <- CM %>% mutate(decimalLatitude = as.numeric(decimalLatitude),
  decimalLongitude= as.numeric(decimalLongitude), year = as.numeric(year))
```

5 Data Preparation

5.1 Filter Source Data

Extract rows from the comments table (CM) that require action:

```
# Table1: rows with changes or exclusion requests
# Duplicate: "0" = exclude, "1" = keep, NA = no action needed
# Columns with "." prefix contain proposed changes
```

```
table1_raw <- edits.Ar %>%
  select(
    id_CC,
    # Duplicate,
    decimalLatitude,
    decimalLongitude,
    canonicalName,
    year,
    eventDate,
    locality,
    samplingProtocol,
    individualCount,
```



```

        catalogNumber,
        datasetName,
        bibliographicCitation,
        institutionCode,
        occurrenceID,
        Comment,
        starts_with(".")
    )

cat("Table1 rows requiring action:", nrow(table1_raw), "\n")

```

```
## Table1 rows requiring action: 347
```

Note: There are several datasets that are not carried forward because no changes need to be made. These are:

```

# Table2: consolidated master table after de-duplication
table2_raw <- DB.submitted %>%
  select(any_of(c("id_CC", colnames(table1_raw))))

cat("Table2 total rows:", nrow(table2_raw), "\n")

```

```
## Table2 total rows: 7188
```

```

# Verify id_CC is unique
if (any(duplicated(table2_raw$id_CC))) {
  stop("ERROR: id_CC in table2 is not unique!")
}

```

5.2 Standardization Function

Reduce meaningless variation in key fields:

```

standardize_for_matching <- function(df) {
  df %>%
    mutate(
      # Coordinates: ensure numeric (no rounding to preserve precision)
      decimalLatitude = as.numeric(decimalLatitude),
      decimalLongitude = as.numeric(decimalLongitude),

      # Event date: extract YYYY-MM-DD format
      eventDate = if_else(
        is.na(eventDate),
        NA_character_,
        str_sub(as.character(eventDate), 1L, 10L)
      ),

      # Year: from year column or extract from eventDate
      year = {
        y1 <- suppressWarnings(as.integer(year))
        y2 <- suppressWarnings(as.integer(str_extract(eventDate, "\\d{4}")))
        coalesce(y1, y2)
      }
    )
}

```

```

# Bibliographic citation: first 20 chars (avoid minor variations)
bibliographicCitation = str_sub(
  str_trim(as.character(bibliographicCitation)),
  1, 20
),

# Text fields: trim, lowercase, standardize NAs
across(
  c(canonicalName, locality, datasetName, catalogNumber,
    occurrenceID, institutionCode),
  ~ {
    x <- str_to_lower(str_trim(as.character(.x)))
    x[x %in% c("", "na", "n a", "n/a")] <- NA_character_
    x
  }
),

# Dataset name: standardize known variations
datasetName = case_when(
  datasetName %in% c("subpesca-1", "subpesca-2",
    "fish database correa-boisjoly",
    "fishgis_metadatabase_ccorrea") ~ datasetName,
  TRUE ~ str_sub(datasetName, 1L, 4L)
)
}

```

5.3 Apply Standardization

```

table1 <- standardize_for_matching(table1_raw)
table2 <- standardize_for_matching(table2_raw)

cat("Standardization complete.\n")

```

```
## Standardization complete.
```

```
cat("Table1:", nrow(table1), "rows\n")
```

```
## Table1: 347 rows
```

```
cat("Table2:", nrow(table2), "rows\n")
```

```
## Table2: 7188 rows
```

6 Matching Strategy

6.1 Define Matching Keys

Keys are ordered by reliability:

```

# Core keys: most reliable identifiers
core_keys <- c(
  "datasetName",
  "canonicalName",
  "decimalLatitude",
  "decimalLongitude",
  "locality",
  "bibliographicCitation")

# Extra keys: used to resolve ambiguities
extra_keys <- c(
  "catalogNumber",
  "year",
  "locality",
  "samplingProtocol",
  "individualCount",
  "eventDate",
  "institutionCode",
  "occurrenceID")

all_keys <- c(core_keys, extra_keys)

cat("Matching will use up to", length(all_keys), "keys in hierarchical order\n")

```

```
## Matching will use up to 14 keys in hierarchical order
```

```
cat("Core keys:", length(core_keys), "\n")
```

```
## Core keys: 6
```

```
cat("Extra keys:", length(extra_keys), "\n")
```

```
## Extra keys: 8
```

6.2 Strictly Conservative Matching Function

This function adds keys incrementally until a unique match is found. If no unique match is found, no assignment is done, and the `match_id` field is left blank:

```

match_tables_strict <- function(table1, table2, core_keys, extra_keys) {

  # Validate inputs
  if (!"id_CC" %in% names(table2)) {
    stop("table2 must contain unique identifier 'id_CC'")
  }

  if (any(duplicated(table2$id_CC))) {
    stop("id_CC in table2 must be unique")
  }

  # Initialize result
  result <- table1 %>%
    mutate(

```

```

    row_id = row_number(),
    match_id = NA_integer_,
    match_keys_used = NA_character_,
    n_keys_used = NA_integer_,
    n_candidates_final = NA_integer_,
    match_status = NA_character_
  )

all_keys <- c(core_keys, extra_keys)
n_core <- length(core_keys)

# Process each row of table1
for (i in seq_len(nrow(table1))) {

  row_i <- table1[i, , drop = FALSE]

  # Start with ALL rows from table2 (no tracking of matched rows)
  candidates <- table2

  keys_used <- character(0)
  keys_available <- character(0)

  # Identify which keys have non-NA values in this table1 row
  for (key in all_keys) {
    if (key %in% names(row_i) && !is.na(row_i[[key]])) {
      keys_available <- c(keys_available, key)
    }
  }

  # Incrementally add keys until unique match or keys exhausted
  for (key in keys_available) {

    val <- row_i[[key]]

    # Filter candidates: exact match on this key, non-NA in table2
    candidates <- candidates %>%
      filter(!is.na(.data[[key]]), .data[[key]] == val)

    keys_used <- c(keys_used, key)

    # Stop if no candidates remain
    if (nrow(candidates) == 0) {
      result$match_status[i] <- "no_match"
      result$n_candidates_final[i] <- 0
      break
    }

    # Stop if unique match achieved
    if (nrow(candidates) == 1) {
      result$match_id[i] <- candidates$id_CC[1]
      result$match_keys_used[i] <- paste(keys_used, collapse = ";")
      result$n_keys_used[i] <- length(keys_used)
      result$n_candidates_final[i] <- 1
      result$match_status[i] <- "matched"
      break
    }
  }
}

```

```

}

# If we exhausted all keys without achieving uniqueness
if (is.na(result$match_status[i]) && nrow(candidates) > 1) {
  result$match_status[i] <- "ambiguous"
  result$n_candidates_final[i] <- nrow(candidates)
  result$match_keys_used[i] <- paste(keys_used, collapse = ";")
  result$n_keys_used[i] <- length(keys_used)
}

# If no keys were available for this row
if (length(keys_available) == 0) {
  result$match_status[i] <- "no_keys_available"
  result$n_candidates_final[i] <- nrow(table2)
}
}

# Classify match quality based on which keys were used
result <- result %>%
  mutate(
    n_core_keys_used = if_else(
      !is.na(match_keys_used),
      sapply(strsplit(match_keys_used, ";"), function(keys) {
        sum(keys %in% core_keys)
      }),
      NA_integer_
    ),
    match_quality = case_when(
      match_status != "matched" ~ match_status,
      n_core_keys_used == n_core ~ "excellent_all_core",
      n_core_keys_used >= 4 ~ "good_4plus_core",
      n_core_keys_used == 3 ~ "acceptable_3_core",
      n_core_keys_used == 2 ~ "weak_2_core",
      n_core_keys_used <= 1 ~ "very_weak_1_core",
      TRUE ~ "matched"
    )
  )

result %>% select(-row_id)
}

```

6.3 Execute Matching

```
cat("Starting strictly conservative matching...\n")
```

```
## Starting strictly conservative matching...
```

```
cat("Each table1 row will be matched independently against ALL table2 rows.\n\n")
```

```
## Each table1 row will be matched independently against ALL table2 rows.
```

```
table1_matched <- match_tables_strict(
  table1 = table1,
  table2 = table2,
  core_keys = core_keys,
  extra_keys = extra_keys
)

cat("Matching complete.\n")
```

```
## Matching complete.
```

7 Results and Diagnostics

7.1 Overall Matching Success

```
summary_stats <- table1_matched %>%
  summarise(
    total_rows = n(),
    matched = sum(match_status == "matched", na.rm = TRUE),
    ambiguous = sum(match_status == "ambiguous", na.rm = TRUE),
    no_match = sum(match_status == "no_match", na.rm = TRUE),
    no_keys = sum(match_status == "no_keys_available", na.rm = TRUE),
    match_rate = round(100 * matched / total_rows, 1)
  )

kable(summary_stats, caption = "Overall Matching Results")
```

Table 2: Overall Matching Results

total_rows	matched	ambiguous	no_match	no_keys	match_rate
347	108	0	239	0	31.1

7.2 Match Status Distribution

```
status_summary <- table1_matched %>%
  count(match_status) %>%
  mutate(
    percentage = round(100 * n / sum(n), 1)
  ) %>%
  arrange(desc(n))

kable(status_summary, caption = "Match Status Breakdown")
```

Table 3: Match Status Breakdown

match_status	n	percentage
no_match	239	68.9
matched	108	31.1

match_status	n	percentage
--------------	---	------------

```
table1_matched %>%
  count(match_status) %>%
  mutate(match_status = forcats::fct_reorder(match_status, n)) %>%
  ggplot(aes(x = match_status, y = n, fill = match_status)) +
  geom_col() +
  geom_text(aes(label = n), hjust = -0.2) +
  coord_flip() +
  scale_fill_brewer(palette = "Set2") +
  labs(
    title = "Match Status Distribution",
    x = "Status",
    y = "Number of Rows"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```

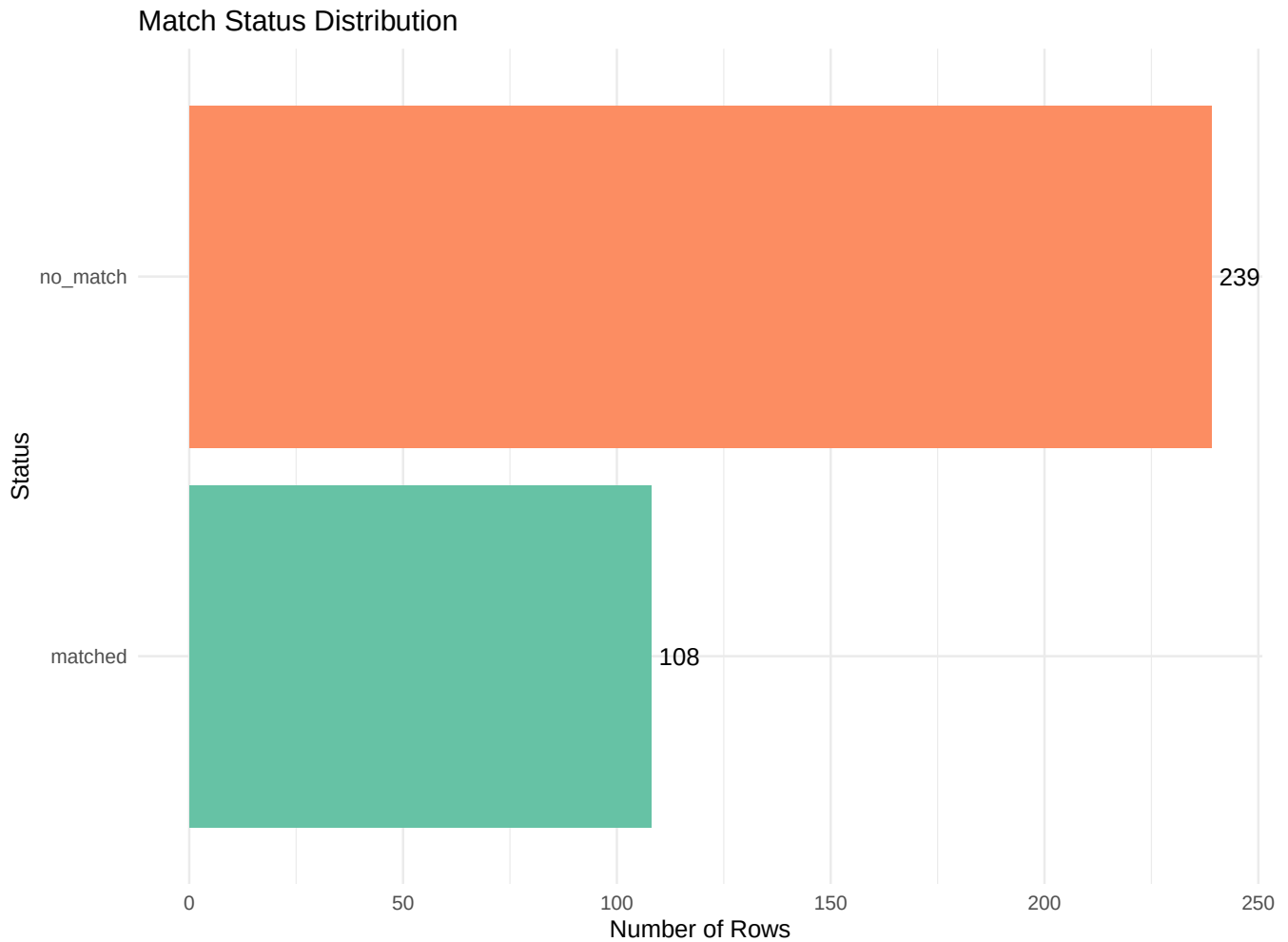


Figure 1: Distribution of Match Status

7.3 Match Quality for Successfully Matched Rows

```
quality_summary <- table1_matched %>%
  filter(match_status == "matched") %>%
  count(match_quality) %>%
  mutate(
    percentage = round(100 * n / sum(n), 1)
  ) %>%
  arrange(desc(n))

kable(quality_summary, caption = "Match Quality Breakdown (Matched Rows Only)")
```

Table 4: Match Quality Breakdown (Matched Rows Only)

match_quality	n	percentage
acceptable_3_core	92	85.2
good_4plus_core	14	13.0
weak_2_core	2	1.9

```
table1_matched %>%
  filter(match_status == "matched") %>%
  count(match_quality) %>%
  mutate(match_quality = forcats::fct_reorder(match_quality, n)) %>%
  ggplot(aes(x = match_quality, y = n, fill = match_quality)) +
  geom_col() +
  geom_text(aes(label = n), hjust = -0.2) +
  coord_flip() +
  scale_fill_brewer(palette = "Set3") +
  labs(
    title = "Match Quality Distribution",
    subtitle = "For successfully matched rows only",
    x = "Quality Category",
    y = "Number of Matches"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```

7.4 Keys Required for Matching

```
keys_summary <- table1_matched %>%
  filter(match_status == "matched") %>%
  count(n_keys_used, n_core_keys_used) %>%
  arrange(n_keys_used, desc(n_core_keys_used))

kable(keys_summary, caption = "Number of Keys Required to Achieve Uniqueness")
```

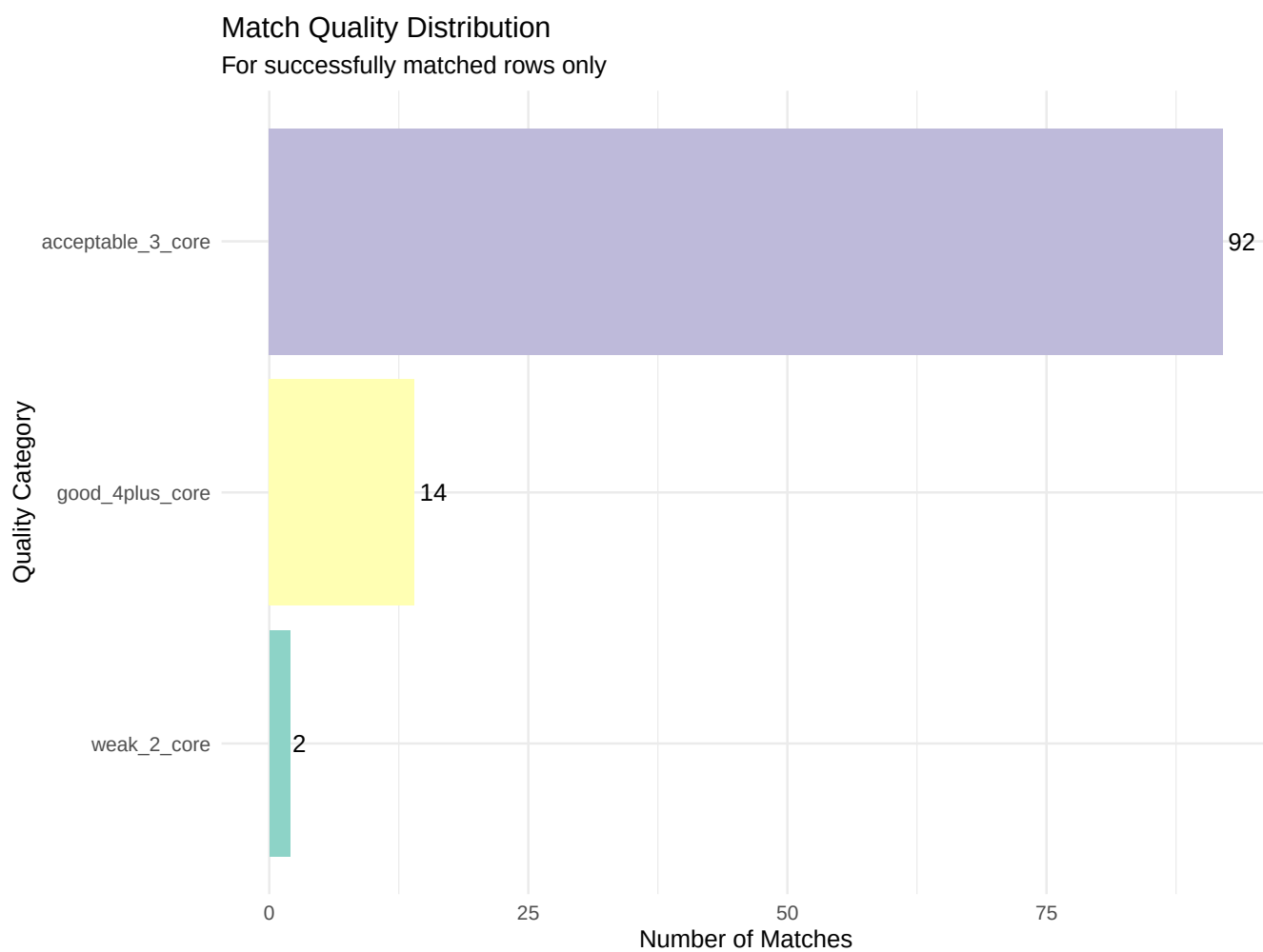



Figure 2: Distribution of Match Quality

Table 5: Number of Keys Required to Achieve Uniqueness

n_keys_used	n_core_keys_used	n
2	2	2
3	3	92
4	4	13
5	4	1

```
table1_matched %>%
  filter(match_status == "matched") %>%
  ggplot(aes(x = n_keys_used, fill = match_quality)) +
  geom_histogram(binwidth = 1, color = "black") +
  scale_fill_brewer(palette = "Set3") +
  labs(
    title = "Number of Keys Required for Unique Match",
    x = "Number of Keys Used",
    y = "Frequency",
    fill = "Match Quality"
  ) +
  theme_minimal()
```

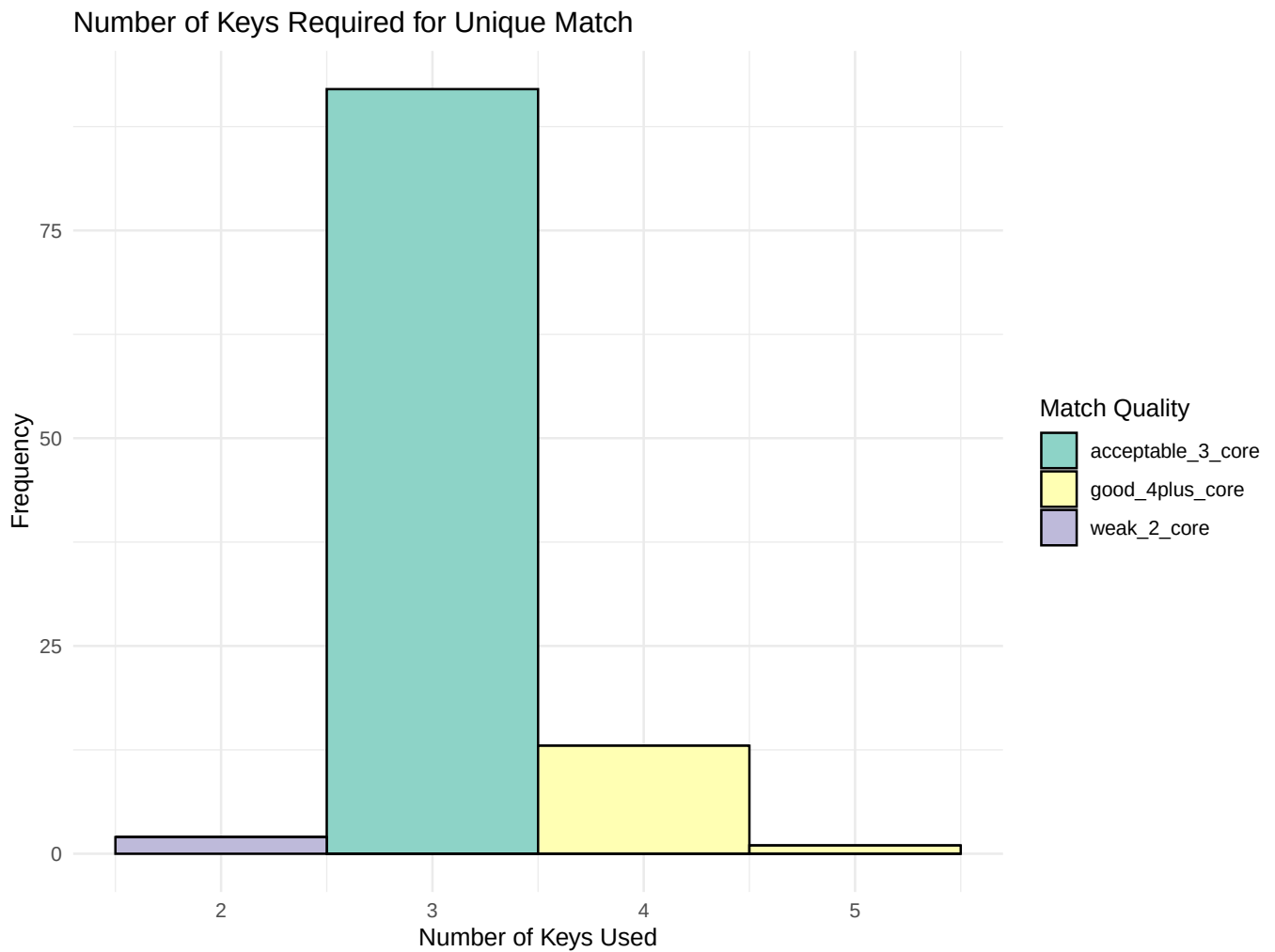


Figure 3: Distribution of Keys Used

7.5 Specific Key Combinations

Top 20 most common key combinations used:

```
key_combos <- table1_matched %>%  
  filter(match_status == "matched") %>%  
  count(match_keys_used, sort = TRUE) %>%  
  head(20)  
  
kable(key_combos, caption = "Most Common Key Combinations")
```

Table 6: Most Common Key Combinations

match_keys_used	n
datasetName;canonicalName;decimalLatitude	92
datasetName;canonicalName;decimalLatitude;decimalLongitude	13
datasetName;canonicalName	2
datasetName;canonicalName;decimalLatitude;decimalLongitude;catalogNumber	1

7.6 Core vs Extra Keys Usage

```
core_extra <- table1_matched %>%  
  filter(match_status == "matched") %>%  
  mutate(  
    n_extra_keys_used = n_keys_used - n_core_keys_used  
  ) %>%  
  count(n_core_keys_used, n_extra_keys_used) %>%  
  arrange(desc(n))  
  
kable(core_extra, caption = "Core vs Extra Keys Usage")
```

Table 7: Core vs Extra Keys Usage

n_core_keys_used	n_extra_keys_used	n
3	0	92
4	0	13
2	0	2
4	1	1

```
table1_matched %>%  
  filter(match_status == "matched") %>%  
  mutate(n_extra_keys_used = n_keys_used - n_core_keys_used) %>%  
  count(n_core_keys_used, n_extra_keys_used) %>%  
  ggplot(aes(x = n_core_keys_used, y = n_extra_keys_used, size = n)) +  
  geom_point(alpha = 0.6, color = "steelblue") +  
  scale_size_continuous(range = c(3, 15)) +  
  labs(  
    title = "Core vs Extra Keys Required",  
    x = "Number of Core Keys Used",  
    y = "Number of Extra Keys Used",  
    size = "Frequency"
```

```
) +  
theme_minimal()
```

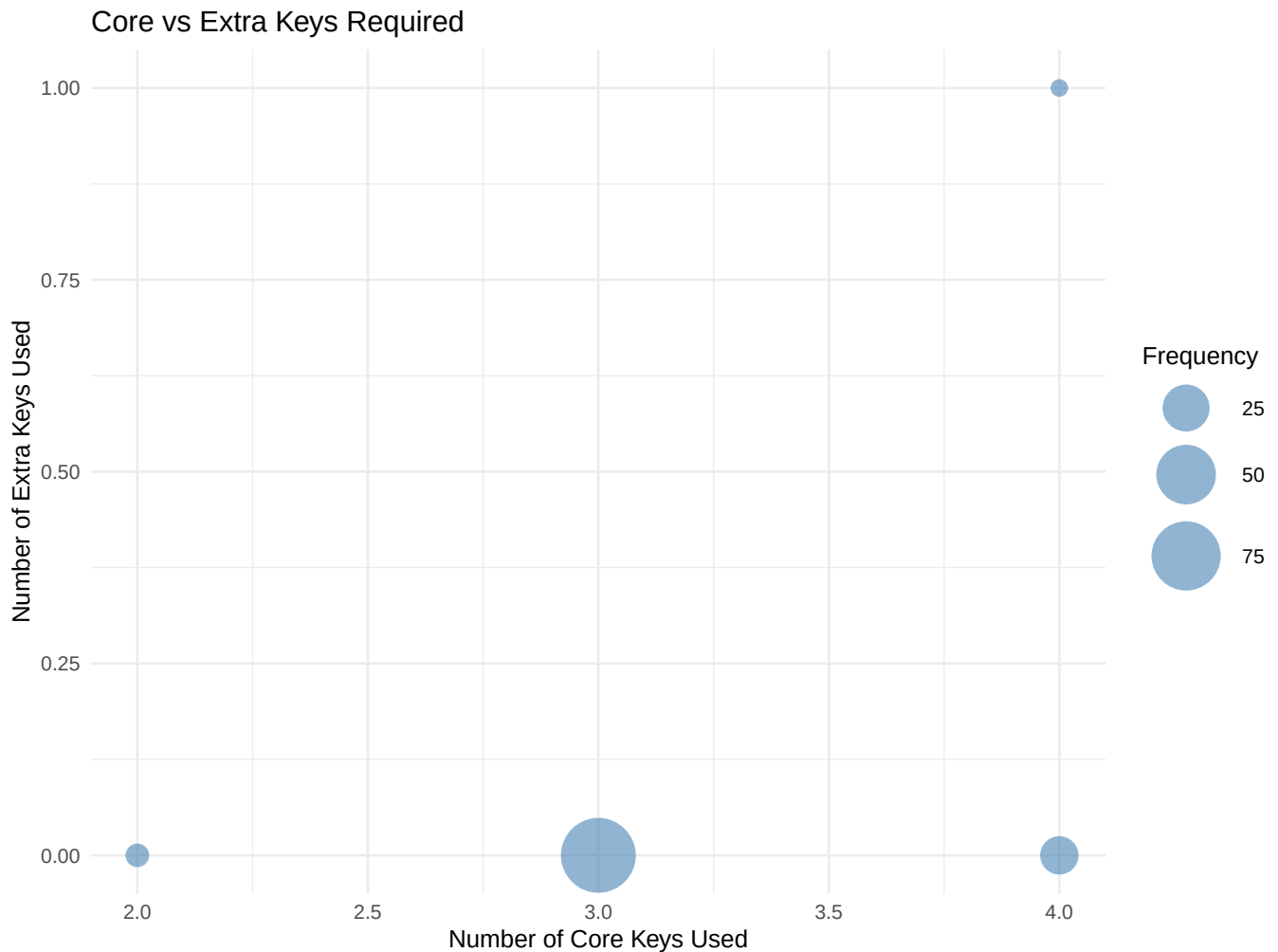


Figure 4: Core vs Extra Keys Scatter

7.7 Uniqueness Validation

Critical check: verify each match_id appears only once:

```
duplicate_matches <- table1_matched %>%  
  filter(match_status == "matched") %>%  
  count(match_id, name = "frequency") %>%  
  filter(frequency > 1)  
  
if (nrow(duplicate_matches) > 0) {  
  cat(" ERROR:", nrow(duplicate_matches), "table2 rows matched multiple times!\n")  
  kable(head(duplicate_matches, 20), caption = "Duplicate Matches (CRITICAL ERROR)")  
} else {  
  cat(" SUCCESS: All matches are unique (one-to-one)\n")  
  cat("Each match_id appears exactly once.\n")  
}
```

```
## SUCCESS: All matches are unique (one-to-one)
## Each match_id appears exactly once.
```

```
table1_matched %>%
  filter(match_status == "matched") %>%
  count(match_id, name = "freq") %>%
  ggplot(aes(x = freq)) +
  geom_histogram(binwidth = 1, color = "black", fill = "steelblue") +
  geom_vline(xintercept = 1, color = "red", linetype = "dashed", size = 1) +
  annotate("text", x = 1.5, y = Inf, label = "All should be at 1",
    vjust = 2, color = "red", fontface = "bold") +
  labs(
    title = "Distribution of match_id Frequency",
    subtitle = "Perfect one-to-one matching requires all frequencies = 1",
    x = "Times each match_id appears",
    y = "Count"
  ) +
  theme_minimal()
```

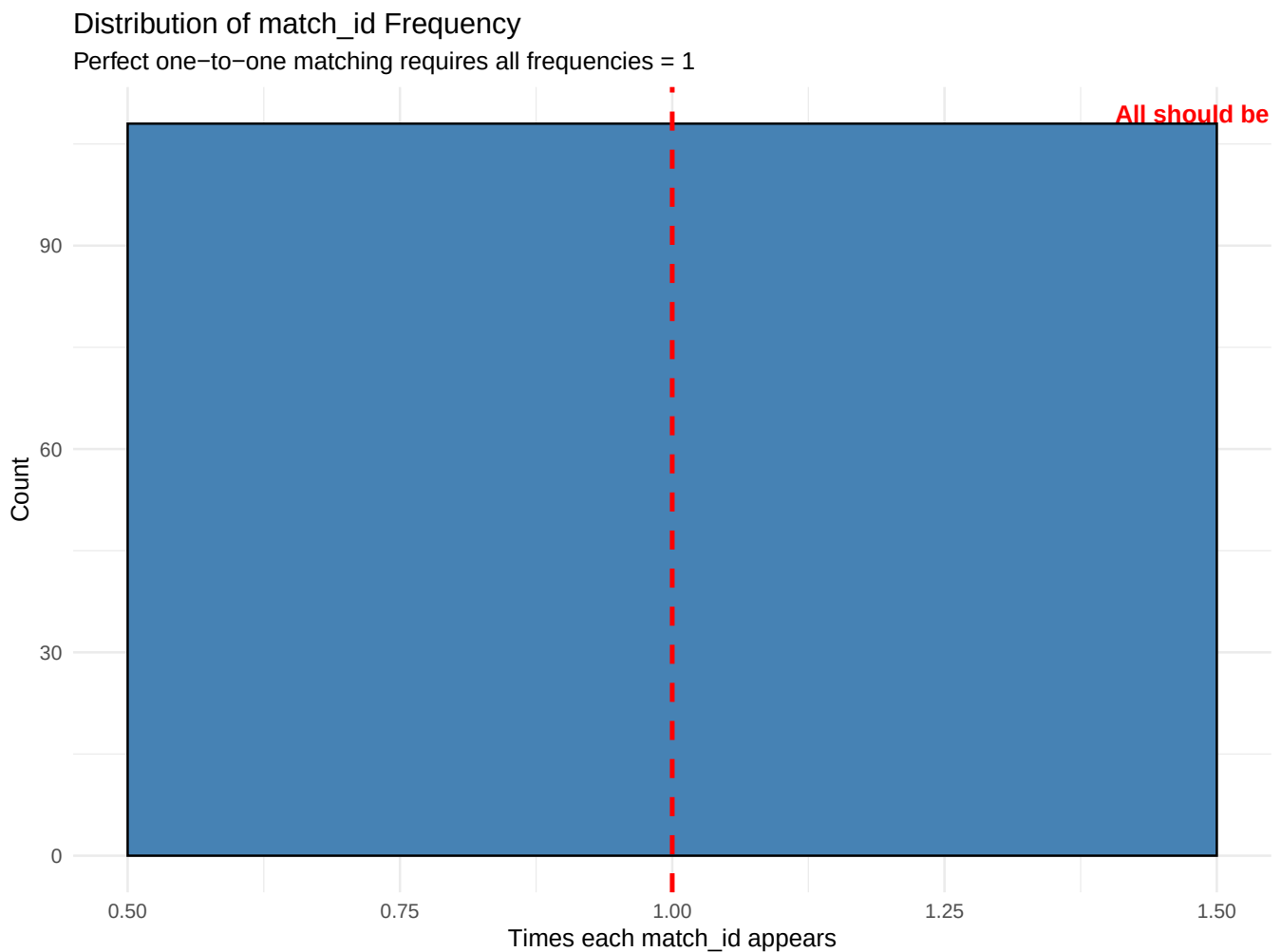


Figure 5: Match ID Frequency Distribution

7.8 Ambiguous Matches Analysis

Rows that had multiple candidates even after using all available keys:

```
if (sum(table1_matched$match_status == "ambiguous", na.rm = TRUE) > 0) {

  ambiguous_summary <- table1_matched %>%
    filter(match_status == "ambiguous") %>%
    group_by(n_candidates_final) %>%
    summarise(
      count = n(),
      avg_keys_used = round(mean(n_keys_used, na.rm = TRUE), 1)
    ) %>%
    arrange(desc(count))

  kable(ambiguous_summary, caption = "Ambiguous Matches: Number of Final Candidates")

  # Show examples of ambiguous matches
  ambiguous_examples <- table1_matched %>%
    filter(match_status == "ambiguous") %>%
    select(
      id_CC, datasetName, canonicalName, year,
      n_candidates_final, n_keys_used, match_keys_used
    ) %>%
    head(10)

  kable(ambiguous_examples, caption = "Examples of Ambiguous Matches")
} else {
  cat("No ambiguous matches found.\n")
}
```

No ambiguous matches found.

7.9 No Match Analysis

Rows where no table2 row matched the criteria:

```
if (sum(table1_matched$match_status == "no_match", na.rm = TRUE) > 0) {

  no_match_summary <- table1_matched %>%
    filter(match_status == "no_match") %>%
    count(datasetName) %>%
    arrange(desc(n))

  kable(no_match_summary, caption = "No Match Cases by Dataset")
} else {
  cat("All rows either matched or were ambiguous.\n")
}
```

Table 8: No Match Cases by Dataset

datasetName	n
agui	230
inat	8
idig	1

7.10 Unmatched Rows Requiring Review

Exclude expected unmatched categories:

```
unmatched_review <- table1_matched %>%
  filter(
    match_status != "matched",
  ) %>%
  select(
    id_CC,
    match_status,
    match_quality,
    n_candidates_final,
    n_keys_used,
    match_keys_used,
    datasetName,
    canonicalName,
    year,
    eventDate,
    locality,
    decimalLatitude,
    decimalLongitude,
    bibliographicCitation,
    catalogNumber,
    occurrenceID,
    institutionCode
  ) %>%
  arrange(match_status, datasetName, canonicalName, year)

cat("Unmatched rows requiring manual review:", nrow(unmatched_review), "\n\n")
```

```
## Unmatched rows requiring manual review: 239
```

```
unmatched_by_status <- unmatched_review %>%
  count(match_status, datasetName) %>%
  arrange(match_status, desc(n))

kable(unmatched_by_status, caption = "Unmatched Rows by Status and Dataset")
```

Table 9: Unmatched Rows by Status and Dataset

match_status	datasetName	n
no_match	agui	230
no_match	inat	8
no_match	idig	1

7.11 Matched, rowwise field-by-Field Comparison

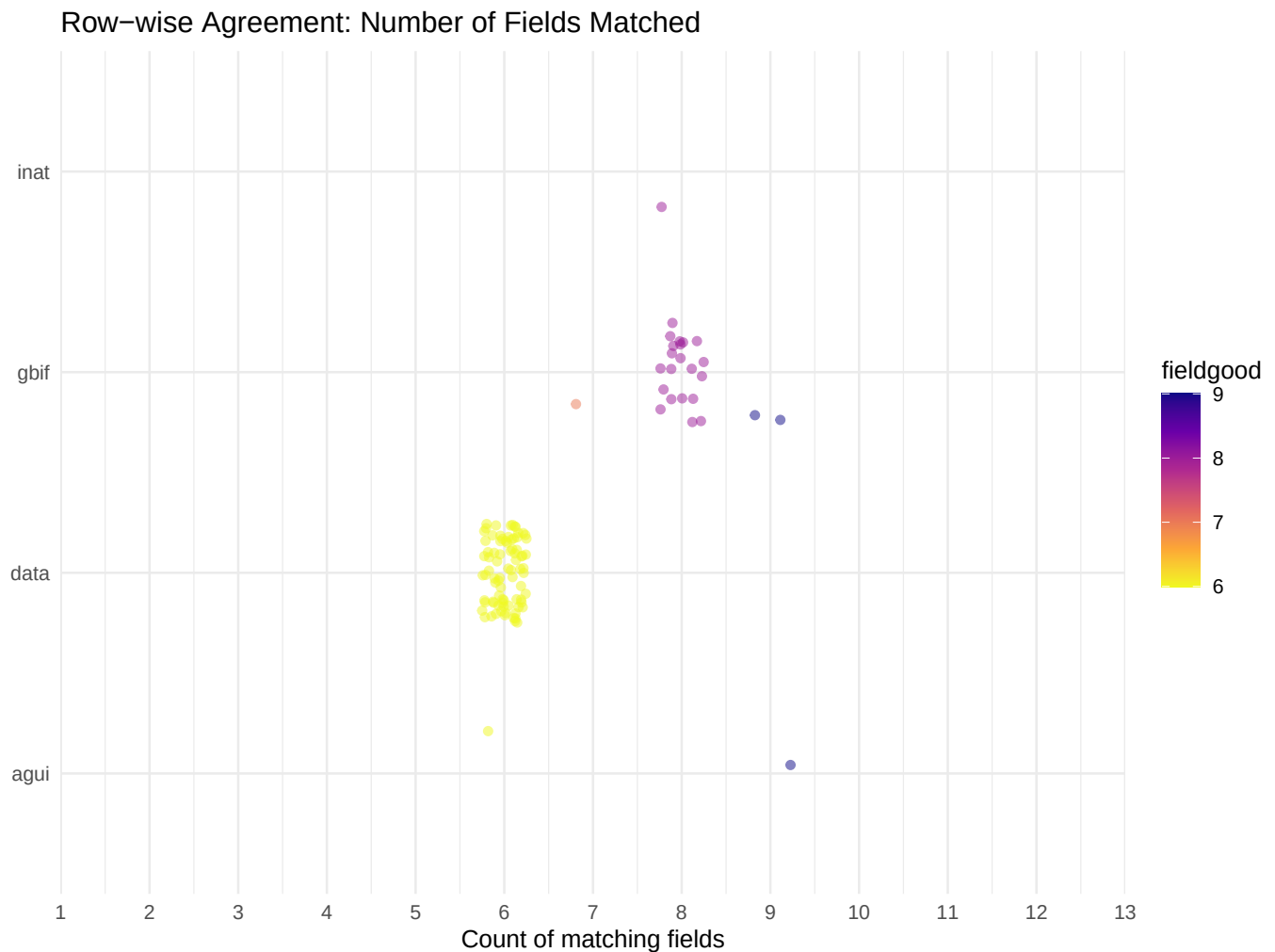
```
if (sum(table1_matched$match_status == "matched", na.rm = TRUE) > 0) {  
  
  # matched subsets  
  tb1 <- table1_matched %>%  
    filter(match_status == "matched") %>%  
    select(  
      match_id,  
      any_of(setdiff(colnames(table2), c("id_CC", starts_with("."))))  
    ) %>%  
    arrange(match_id)  
  
  tb2 <- table2 %>%  
    filter(id_CC %in% tb1$match_id) %>%  
    arrange(id_CC)  
  
  tb2 <- tb2[match(tb1$match_id, tb2$id_CC), ]  
  
  comparison_cols <- setdiff(intersect(names(tb1), names(tb2)), "match_id")  
  
  diff.strict <- tb1[, comparison_cols] == tb2[, comparison_cols]  
  
  tmp <- table1_matched %>%  
    filter(match_status == "matched") %>%  
    select(  
      id_CC_old = id_CC,  
      match_id,  
      match_keys_used,  
      dataset = datasetName  
    ) %>%  
    arrange(match_id)  
  
  # logical match matrix -> data frame  
  diff.strict <- as.data.frame(diff.strict)  
  
  # add metadata for each matched row  
  diff.strict <- bind_cols(  
    tmp[!is.na(tmp$match_id), ], # keep only matched rows in tmp  
    diff.strict  
  )  
  
  # compute number of matching fields row-wise  
  diff.strict <- diff.strict %>%  
    mutate(fieldgood = rowSums(across(!c(id_CC_old, match_id, match_keys_used, dataset)), na.rm = TRUE))  
  
  # plot  
  ggplot(diff.strict, aes(x = dataset, y = fieldgood)) +  
    geom_jitter(aes(color = fieldgood),  
               height = 0.25, width = 0.25, alpha = 0.5) +  
    coord_flip() +  
    scale_y_continuous(breaks = seq(1, length(comparison_cols)),  
                       limits = c(1, length(comparison_cols)),  
                       expand = FALSE) +  
    scale_color_viridis_c(option = "plasma", direction = -1) +
```



```

labs(
  title = "Row-wise Agreement: Number of Fields Matched",
  x = NULL,
  y = "Count of matching fields"
) +
theme_minimal()
} else {
  cat("No matched rows to compare.\n")
}

```



```

if (sum(table1_matched$match_status == "matched", na.rm = TRUE) > 0) {
  # matched subsets
  tb1 <- table1_matched %>%
    filter(match_status == "matched") %>%
    select(
      match_id,
      any_of(setdiff(colnames(table2), c("id_CC", starts_with("."))))
    ) %>%
    arrange(match_id)

  tb2 <- table2 %>%

```

```

  filter(id_CC %in% tb1$match_id) %>%
  arrange(id_CC)

tb2 <- tb2[match(tb1$match_id, tb2$id_CC), ]

comparison_cols <- setdiff(intersect(names(tb1), names(tb2)), "match_id")

# COUNT DIFFERENCES THIS TIME
diff.strict2 <- tb1[, comparison_cols] != tb2[, comparison_cols]

tmp <- table1_matched %>%
  filter(match_status == "matched") %>%
  select(
    id_CC_old = id_CC,
    match_id,
    match_keys_used,
    dataset = datasetName
  ) %>%
  arrange(match_id)

# logical match matrix -> data frame
diff.strict2 <- as.data.frame(diff.strict2)

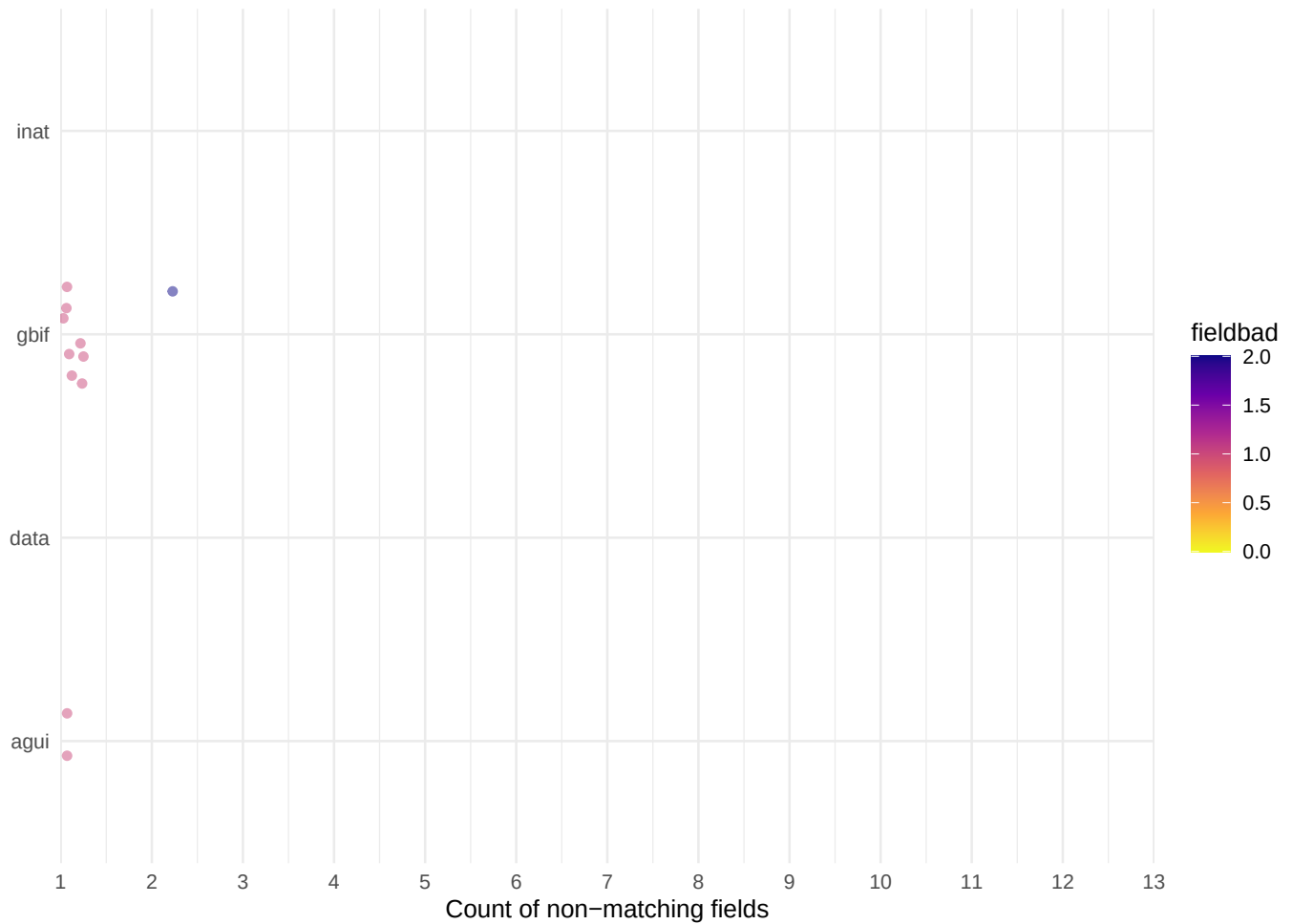
# add metadata for each matched row
diff.strict2 <- bind_cols(
  tmp[!is.na(tmp$match_id), ], # keep only matched rows in tmp
  diff.strict2
)

# compute number of matching fields row-wise
diff.strict.falses <- diff.strict2 %>%
  mutate(fieldbad = rowSums(across(!c(id_CC_old, match_id, match_keys_used, dataset)), na.rm = TRUE))

# plot
ggplot(diff.strict.falses, aes(x = dataset, y = fieldbad)) +
  geom_jitter(aes(color = fieldbad),
    height = 0.25, width = 0.25, alpha = 0.5) +
  coord_flip() +
  scale_y_continuous(breaks = seq(1, length(comparison_cols)),
    limits = c(1, length(comparison_cols)),
    expand = FALSE) +
  scale_color_viridis_c(option = "plasma", direction = -1) +
  labs(
    title = "Row-wise Agreement: Number of Fields Matched",
    x = NULL,
    y = "Count of non-matching fields"
  ) +
  theme_minimal()
} else {
  cat("No matched rows to compare.\n")
}

```

Row-wise Agreement: Number of Fields Matched



```
good.and.bad.matches <- diff.strict %>% full_join(diff.strict.falses, by = "id_CC_old", keep = T) %>%
  select(dataset.x, id_CC_old.x, match_id.x, fieldgood, fieldbad) %>%
  mutate(logodd.ratio = log10((fieldgood + 1) / (fieldbad + 1)),
         precent.good = 100 * (fieldgood /length(comparison_cols)),
         percent.bad = 100 * (fieldbad /length(comparison_cols)))

good.and.bad.matches %>% filter(fieldbad >= 3) %>%
  kable(caption = "Matched rows with 3 or more mismatches.")
```

Table 10: Matched rows with 3 or more mismatches.

dataset.x	id_CC_old.x	match_id.x	fieldgood	fieldbad	logodd.ratio	precent.good	percent.bad
-----------	-------------	------------	-----------	----------	--------------	--------------	-------------

```
# write.csv(
#   good.and.bad.matches,
#   "./outputs_global_analyses/Table1_good-bad_matches_Argentina.csv",
#   fileEncoding = "UTF-8",
#   row.names = FALSE
# )
```

```

good.and.bad.matches %>%
ggplot(aes(x = dataset.x, y = logodd.ratio)) +
  geom_jitter(aes(color = logodd.ratio),
              height = 0.25, width = 0.25, alpha = 0.5) +
  coord_flip() +
  # scale_y_continuous(breaks = seq(1, length(comparison_cols)),
  #                   limits= c(1, length(comparison_cols)),
  #                   expand = FALSE) +
  scale_color_viridis_c(option = "plasma", direction = -1) +
  labs(
    title = "Row-wise Agreement: Log-Ratio of matched-to-unmatched fields",
    x = NULL,
    y = "Log of the odds"
  ) +
  theme_minimal()

```



7.12 Matched, overall field-by-Field Comparison

For successfully matched rows, compare original values:

```

if (sum(table1_matched$match_status == "matched", na.rm = TRUE) > 0) {

```

```

# Prepare matched subsets
tb1 <- table1_matched %>%
  filter(match_status == "matched") %>%
  select(
    match_id,
    any_of(setdiff(colnames(table2), c("id_CC", starts_with("."))))
  ) %>%
  arrange(match_id)

tb2 <- table2 %>%
  filter(id_CC %in% tb1$match_id) %>%
  arrange(id_CC)

# Ensure same order
tb2 <- tb2[match(tb1$match_id, tb2$id_CC), ]

# Compare field by field
comparison_cols <- intersect(names(tb1), names(tb2))
comparison_cols <- setdiff(comparison_cols, "match_id")

agreement <- sapply(comparison_cols, function(col) {
  sum(tb1[[col]] == tb2[[col]], na.rm = TRUE)
})

total <- nrow(tb1)

agreement_summary <- data.frame(
  field = comparison_cols,
  agreements = agreement,
  total = total,
  agreement_rate = round(100 * agreement / total, 1)
) %>%
  arrange(desc(agreement_rate))

kable(agreement_summary, caption = "Field Agreement Rates for Matched Rows")
} else {
  cat("No matched rows to compare.\n")
}

```

Table 11: Field Agreement Rates for Matched Rows

	field	agreements	total	agreement_rate
decimalLatitude	decimalLatitude	108	108	100.0
canonicalName	canonicalName	108	108	100.0
datasetName	datasetName	108	108	100.0
decimalLongitude	decimalLongitude	105	108	97.2
institutionCode	institutionCode	105	108	97.2
occurrenceID	occurrenceID	105	108	97.2
catalogNumber	catalogNumber	26	108	24.1
year	year	21	108	19.4
locality	locality	10	108	9.3
individualCount	individualCount	3	108	2.8
bibliographicCitation	bibliographicCitation	2	108	1.9
samplingProtocol	samplingProtocol	1	108	0.9

	field	agreements	total	agreement_rate
eventDate	eventDate	0	108	0.0

```

if (sum(table1_matched$match_status == "matched", na.rm = TRUE) > 0) {
  agreement_summary %>%
    mutate(field = forcats::fct_reorder(field, agreement_rate)) %>%
    ggplot(aes(x = field, y = agreement_rate, fill = agreement_rate)) +
    geom_col() +
    geom_hline(yintercept = 100, linetype = "dashed", color = "red") +
    coord_flip() +
    scale_fill_gradient(low = "orange", high = "darkgreen") +
    labs(
      title = "Field Agreement Rates",
      subtitle = "Comparison between matched table1 and table2 rows",
      x = "Field",
      y = "Agreement Rate (%)",
      fill = "Agreement %"
    ) +
    theme_minimal()
}

```

```

if (sum(table1_matched$match_status == "matched", na.rm = TRUE) > 0) {

  # Prepare matched subsets, keep datasetName
  tb1 <- table1_matched %>%
    filter(match_status == "matched") %>%
    select(
      datasetName,          # keep for stratification
      match_id,
      any_of(setdiff(colnames(table2), c("id_CC", starts_with("."))))
    ) %>%
    arrange(match_id)

  tb2 <- table2 %>%
    filter(id_CC %in% tb1$match_id) %>%
    arrange(id_CC)

  # Ensure same order
  tb2 <- tb2[match(tb1$match_id, tb2$id_CC), ]

  # Columns to compare
  comparison_cols <- intersect(names(tb1), names(tb2))
  comparison_cols <- setdiff(comparison_cols, c("match_id", "datasetName"))

  # Logical matrix of equalities for each field
  equal_df <- purrr::map_dfc(
    comparison_cols,
    ~ tb1[[.x]] == tb2[[.x]]
  )
  names(equal_df) <- comparison_cols

  # Long format: one row per (datasetName, field, row-pair)
  agreement_summary <- bind_cols(
    datasetName = tb1$datasetName,
    equal_df
  )
}

```

```

) %>%
  pivot_longer(
    cols = -datasetName,
    names_to = "field",
    values_to = "equal"
  ) %>%
  group_by(datasetName, field) %>%
  summarise(
    agreements = sum(equal, na.rm = TRUE),
    total = sum(!is.na(equal)),
    agreement_rate = round(100 * agreements / total, 1),
    .groups = "drop"
  ) %>%
  arrange(datasetName, desc(agreement_rate))

kable(agreement_summary, caption = "Field Agreement Rates for Matched Rows, by datasetName")
} else {
  cat("No matched rows to compare.\n")
}

```

Table 12: Field Agreement Rates for Matched Rows, by dataset-
Name

datasetName	field	agreements	total	agreement_rate
agui	bibliographicCitation	2	2	100.0
agui	canonicalName	2	2	100.0
agui	catalogNumber	2	2	100.0
agui	decimalLatitude	2	2	100.0
agui	individualCount	1	1	100.0
agui	locality	2	2	100.0
agui	samplingProtocol	1	1	100.0
agui	year	1	1	100.0
agui	decimalLongitude	0	2	0.0
agui	eventDate	0	0	NaN
agui	institutionCode	0	0	NaN
agui	occurrenceID	0	0	NaN
data	canonicalName	81	81	100.0
data	decimalLatitude	81	81	100.0
data	decimalLongitude	81	81	100.0
data	institutionCode	81	81	100.0
data	occurrenceID	81	81	100.0
data	bibliographicCitation	0	0	NaN
data	catalogNumber	0	0	NaN
data	eventDate	0	0	NaN
data	individualCount	0	0	NaN
data	locality	0	0	NaN
data	samplingProtocol	0	0	NaN
data	year	0	0	NaN
gbif	canonicalName	24	24	100.0
gbif	catalogNumber	23	23	100.0
gbif	decimalLatitude	24	24	100.0
gbif	individualCount	2	2	100.0
gbif	institutionCode	24	24	100.0
gbif	locality	7	7	100.0

datasetName	field	agreements	total	agreement_rate
gbif	occurrenceID	23	23	100.0
gbif	year	19	19	100.0
gbif	decimalLongitude	23	24	95.8
gbif	eventDate	0	19	0.0
gbif	bibliographicCitation	0	0	NaN
gbif	samplingProtocol	0	0	NaN
inat	canonicalName	1	1	100.0
inat	catalogNumber	1	1	100.0
inat	decimalLatitude	1	1	100.0
inat	decimalLongitude	1	1	100.0
inat	locality	1	1	100.0
inat	occurrenceID	1	1	100.0
inat	year	1	1	100.0
inat	eventDate	0	1	0.0
inat	bibliographicCitation	0	0	NaN
inat	individualCount	0	0	NaN
inat	institutionCode	0	0	NaN
inat	samplingProtocol	0	0	NaN

```

if (sum(table1_matched$match_status == "matched", na.rm = TRUE) > 0) {
  agreement_summary %>%
    mutate(field = forcats::fct_reorder(field, agreement_rate)) %>%
    ggplot(aes(x = field, y = agreement_rate, fill = agreement_rate)) +
    geom_col() +
    geom_hline(yintercept = 100, linetype = "dashed", color = "red") +
    coord_flip() +
    scale_fill_gradient(low = "orange", high = "darkgreen") +
    facet_wrap(~ datasetName) +
    labs(
      title = "Field Agreement Rates by datasetName",
      subtitle = "Comparison between matched table1 and table2 rows",
      x = "Field",
      y = "Agreement Rate (%)",
      fill = "Agreement %"
    ) +
    theme_minimal()
}

```

8 Export Results

8.1 Matched Records

```

table1_export <- table1_matched %>%
  left_join(
    table1_raw %>% select(id_CC, starts_with(".")),
    by = "id_CC"
  ) %>%
  # select(
  #   id_CC_original = id_CC,
  #   match_id,

```

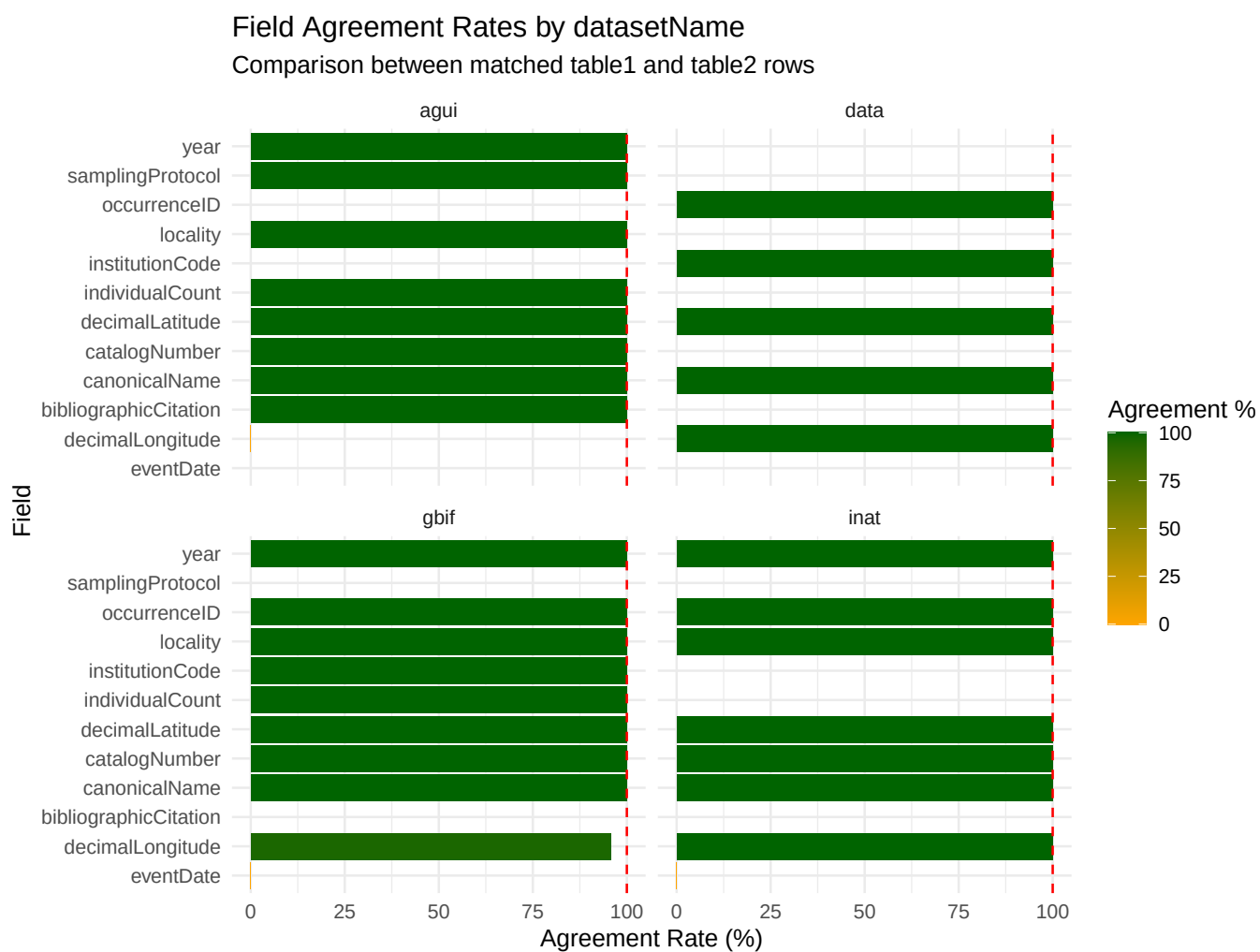



Figure 6: Field Agreement Rates by datasetName

```

# match_status,
# match_quality,
# n_keys_used,
# n_core_keys_used,
# n_candidates_final,
# match_keys_used,
# datasetName,
# canonicalName,
# year,
# eventDate,
# locality,
# decimalLatitude,
# decimalLongitude,
# bibliographicCitation,
# catalogNumber,
# occurrenceID,
# institutionCode,
# Duplicate,
# starts_with(".")
# ) %>%
arrange(match_status, match_quality, datasetName, id_CC)

write.csv(
  table1_export,
  "./outputs_global_analyses/Table1_matched_strict_Argentina.csv",
  fileEncoding = "UTF-8",
  row.names = FALSE
)

cat("Exported", nrow(table1_export), "rows to Table1_matched_strict.csv\n")

```

Exported 347 rows to Table1_matched_strict.csv

8.2 Good-to-bad match ratio

```

write.csv(
  good.and.bad.matches,
  "./outputs_global_analyses/Table1_good-bad_matches_Argentina.csv",
  fileEncoding = "UTF-8", row.names = FALSE, sep=","
)

```

8.3 Unmatched Records for Manual Review

```

if (exists("unmatched_review") && nrow(unmatched_review) > 0) {
  write.csv(
    unmatched_review,
    "./outputs_global_analyses/Table1_unmatched_review.csv",
    fileEncoding = "UTF-8",
    row.names = FALSE
  )
}

```

```
cat("Exported", nrow(unmatched_review), "unmatched rows for review\n")
}
```

8.4 Ambiguous Matches for Investigation

```
ambiguous_export <- table1_matched %>%
  filter(match_status == "ambiguous") %>%
  select(
    id_CC,
    n_candidates_final,
    n_keys_used,
    match_keys_used,
    datasetName,
    canonicalName,
    year,
    locality,
    decimalLatitude,
    decimalLongitude,
    bibliographicCitation,
    catalogNumber,
    occurrenceID,
    institutionCode
  ) %>%
  arrange(desc(n_candidates_final), datasetName)

if (nrow(ambiguous_export) > 0) {
  write.csv(
    ambiguous_export,
    "./outputs_global_analyses/Table1_ambiguous_matches.csv",
    fileEncoding = "UTF-8",
    row.names = FALSE
  )

  cat("Exported", nrow(ambiguous_export), "ambiguous matches for investigation\n")
}
```

9 Summary Statistics Table

```
summary_table <- data.frame(
  Metric = c(
    "Total table1 rows",
    "Successfully matched",
    "Ambiguous (multiple candidates)",
    "No match found",
    "No keys available",
    "Match rate (%)",
    "One-to-one constraint satisfied"
  ),
  Value = c(
    nrow(table1_matched),
    sum(table1_matched$match_status == "matched", na.rm = TRUE),
```

```

    sum(table1_matched$match_status == "ambiguous", na.rm = TRUE),
    sum(table1_matched$match_status == "no_match", na.rm = TRUE),
    sum(table1_matched$match_status == "no_keys_available", na.rm = TRUE),
    round(100 * sum(table1_matched$match_status == "matched", na.rm = TRUE) / nrow(table1_matched), 1),
    ifelse(nrow(duplicate_matches) == 0, "Yes ", "No ")
  )
)

kable(summary_table, caption = "Final Summary Statistics")

```

Table 13: Final Summary Statistics

Metric	Value
Total table1 rows	347
Successfully matched	108
Ambiguous (multiple candidates)	0
No match found	239
No keys available	0
Match rate (%)	31.1
One-to-one constraint satisfied	Yes

```

#
# table1_export[,c("id_CC_original", "match_id", "datasetName", "canonicalName", "year",
# "locality", "decimalLatitude", "decimalLongitude", "bibliographicCitation",
# "occurrenceID", "catalogNumber", "institutionCode")] %>%

```

10 Session Information

```
sessionInfo()
```

```

## R version 4.4.1 (2024-06-14 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26100)
##
## Matrix products: default
##
## locale:
## [1] LC_COLLATE=English_Canada.utf8  LC_CTYPE=English_Canada.utf8
## [3] LC_MONETARY=English_Canada.utf8 LC_NUMERIC=C
## [5] LC_TIME=English_Canada.utf8
##
## time zone: America/Santiago
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] tidyr_1.3.1  knitr_1.50  ggplot2_4.0.1 stringr_1.6.0 dplyr_1.1.4
##

```

```
## loaded via a namespace (and not attached):
## [1] gtable_0.3.6      compiler_4.4.1    tidyselect_1.2.1  tinytex_0.58
## [5] scales_1.4.0      yaml_2.3.10      fastmap_1.2.0     readxl_1.4.5
## [9] R6_2.6.1          labeling_0.4.3    generics_0.1.4    forcats_1.0.1
## [13] tibble_3.3.0      pillar_1.11.1    RColorBrewer_1.1-3 rlang_1.1.6
## [17] stringi_1.8.7     xfun_0.54         S7_0.2.1          viridisLite_0.4.2
## [21] cli_3.6.5         withr_3.0.2       magrittr_2.0.4     digest_0.6.39
## [25] grid_4.4.1        rstudioapi_0.17.1 lifecycle_1.0.4    vctrs_0.6.5
## [29] evaluate_1.0.5    glue_1.8.0        farver_2.1.2       cellranger_1.1.0
## [33] rmarkdown_2.30    purrr_1.2.0       tools_4.4.1        pkgconfig_2.0.3
## [37] htmltools_0.5.8.1
```

11 Conclusion

This strictly conservative matching strategy ensures maximum reliability by:

1. **No tracking of matched table2 rows:** Each table1 row is evaluated independently against the entire table2
2. **Incremental key addition:** Keys are added one-by-one until uniqueness is achieved
3. **Explicit ambiguity detection:** Rows with multiple candidates are flagged as ambiguous
4. **Complete transparency:** All key combinations used are recorded for validation

Final Results:

- **Total processed:** 347 rows
- **Successfully matched:** 108 rows
- **Match rate:** 31.1%
- **One-to-one verified:** Yes

All matched records represent unambiguous, unique links between table1 and table2.